

IaC Provisioning for Agriculture System

(Krishi AI Assistant)

Manufacturing Operational Insights Report

(Mention your Project Title Above)

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Praniti Kshirsagar	EN22CS301726
2	Pawani Shah	EN22CS301700
3	Pranjal Patidar	EN22CS301730
4	Prafulla Kumar Verma	EN22CS301715
5	Ritik Thakur	EN22CS301811

Group Name: Group 08D7

Project Number:DO-31

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Shyam Patel

Academic Year:Jan25-May25

Problem Statement & Objectives

1. Problem Statement

Setting up and managing infrastructure for agricultural systems has traditionally been a manual, error-prone, and time-consuming process. System administrators and DevOps teams face challenges with inconsistent environment configurations across development, staging, and production — leading to deployment failures, dependency conflicts, and difficulty in reproducing environments.

For an AI-powered agricultural support system like Krishi AI Assistant, reliable and repeatable infrastructure provisioning is critical. The absence of Infrastructure as Code (IaC) practices means every new deployment requires manual installation of dependencies (Docker, runtime libraries, API keys), manual server configuration, and significant human intervention — making scalability and maintenance difficult.

There is a clear need to automate the provisioning of infrastructure using IaC tools such as Terraform or Ansible, combined with Docker containerization and CI/CD pipelines, so that the entire agricultural system can be deployed consistently, reliably, and at scale — whether on a local Docker environment or cloud instances such as AWS EC2.

2. Project Objectives

- Automate the provisioning of infrastructure for the Agriculture system using Infrastructure as Code (IaC) tools — eliminating manual configuration and setup errors.
- Use script-based deployment to automatically install core dependencies such as Docker, and configure the runtime environment on local machines or cloud instances (AWS EC2).
- Containerize the full-stack application (React frontend + FastAPI backend) using Docker and Docker Compose for isolated, repeatable deployments.
- Build and automate CI/CD pipelines using Git-action to handle code builds, testing, and deployment without manual intervention.
- Develop an AI-powered Krishi AI Assistant that analyzes crop images and provides disease diagnosis, treatment suggestions, market price estimations, and insurance information using Groq AI APIs.
- Ensure scalable and independently deployable services — frontend, backend, and monitoring — each running as separate containers.
- Use Git and GitHub for version control and collaborative development across the team.

3. Scope of the Project

This project covers the end-to-end DevOps and IaC lifecycle for the Krishi AI Assistant, including:

- Script-based automated installation of Docker and all required dependencies on the target environment (local or AWS EC2).
- Docker-based containerization of frontend (React.js) and backend (FastAPI/Python) services.
- Docker Compose-based orchestration to manage multi-container deployment as a single automated operation.
- CI/CD pipeline automation using Git-action for continuous integration and repeatable deployments.
- Integration with Groq AI APIs (LLM + Vision) for intelligent crop analysis.
- Lightweight SQLite database design with JWT-based authentication for user and request management.
- Container monitoring via Portainer CE.

Proposed Solution

The proposed solution is the Krishi AI Assistant — a fully containerized, AI-powered agricultural support system built on a DevOps-first approach. The core of the solution is automated infrastructure provisioning using Infrastructure as Code principles: instead of manually configuring servers, the entire environment is defined in code (Dockerfiles, Docker Compose, Git-action pipelines, and shell scripts) and can be reproduced reliably on any target machine or cloud instance.

1. Key features

- IaC-based automated provisioning — shell scripts automate the installation of Docker, Docker Compose, and all dependencies on a fresh local or AWS EC2 instance with zero manual steps.
- Docker containerization — each service (frontend, backend, monitoring) is packaged as an independent Docker image ensuring environment consistency.
- CI/CD pipeline via Git-action — automated build, test, and deploy cycle triggered by Git commits.
- AI-powered crop disease detection — Groq Vision API analyzes uploaded crop images and returns disease diagnosis and urgency-level assessment.
- Treatment suggestions and actionable recommendations provided in real time to the farmer.
- Market price estimation and insurance scheme information accessible from the same interface.
- Voice and text input support for improved farmer accessibility.
- JWT-based secure authentication with stateless API design for horizontal scalability.
- Portainer UI for real-time container management and health monitoring.

2. Overall Architecture / Workflow

The system follows a Client-Server architecture with a fully containerized, IaC-driven deployment model:

Layer	Component	Technology
IaC / Provisioning	Automated env setup scripts	Shell Scripts + Docker Compose
CI/CD Pipeline	Automated build & deploy	Git-action + Git/GitHub
Frontend	User Interface Container	React.js — Port 3000 (~63 MB)
Backend	API & Business Logic Container	Python + FastAPI — Port 8000 (~1.03 GB)
AI Layer	External AI Processing	Groq LLM + Vision API
Database	Data Storage	SQLite + SQLAlchemy ORM
Monitoring	Container Management	Portainer CE
Cloud / Hosting	Deployment Target	AWS EC2 (Ubuntu)

Provisioning Flow:

Fresh Server → Run IaC Script → Docker + Dependencies Installed → Docker Compose Up → All Containers Running → Application Live

CI/CD Flow:

Git Push → Git-action Triggered → Build Docker Images → Run Tests → Deploy Containers → Application Updated

Request Flow:

User Browser → Frontend Container (Port 3000) → Backend Container (Port 8000) → Groq AI APIs → Response → User

3. Tools & Technologies Used

Category	Tool / Technology	Purpose
laC / Automation	Shell Scripts + Docker Compose	Automated dependency installation & env setup
Containerization	Docker	Package services as portable images
Orchestration	Docker Compose	Multi-container deployment as code
CI/CD	Git-action	Automated build, test, and deployment pipeline
Version Control	Git / GitHub	Source code management and collaboration
Frontend Framework	React.js	User interface development
Backend Framework	FastAPI (Python)	REST API development
AI / LLM	Groq API (LLM + Vision)	Crop image analysis and AI responses
Database	SQLite + SQLAlchemy ORM	User and request data storage
Authentication	JWT (JSON Web Tokens)	Secure stateless user authentication
Monitoring	Portainer CE	Container management UI
Cloud Platform	AWS EC2 (Ubuntu)	Hosting and cloud deployment

Results & Output

Add the below details here:

1. Screenshots / outputs

Docker container-

```

ubuntu@ip-172-31-39-36:~$ cd ~/Agriculture-Support-Triage-Agent
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
deea2e064e5c   portainer/portainer-ce:latest       "/portainer"            5 days ago    Up 5 days    8000/tcp, 9443/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
portainer
3262f5d343fc   agriculture-support-triage-agent-frontend "/docker-entrypoint..." 8 days ago    Up 5 days    0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
agriculture-support-triage-agent-frontend-1
cb244cd8169d   agriculture-support-triage-agent-backend "unicorn main:app --..." 8 days ago    Up 5 days    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
agriculture-support-triage-agent-backend-1
  
```

Docker images-

```

ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$ docker images
REPOSITORY                                TAG          IMAGE ID       CREATED        SIZE
agriculture-support-triage-agent-frontend  latest      cfc253319e48   8 days ago    63.4MB
<none>                                     <none>       d1d8bcf9a60a   8 days ago    63.4MB
<none>                                     <none>       1d4ce9c64984   8 days ago    63.4MB
agriculture-support-triage-agent-backend   latest      8e1d126b0661   8 days ago    1.03GB
portainer/portainer-ce                    latest      2ad8d683056c   4 weeks ago   183MB
  
```

Portainer with ip address-

```
3.110.128.84:9000/#!/auth
```

PORTAINER.IO

Log in to your account

Welcome back! Please enter your details

Username

Password



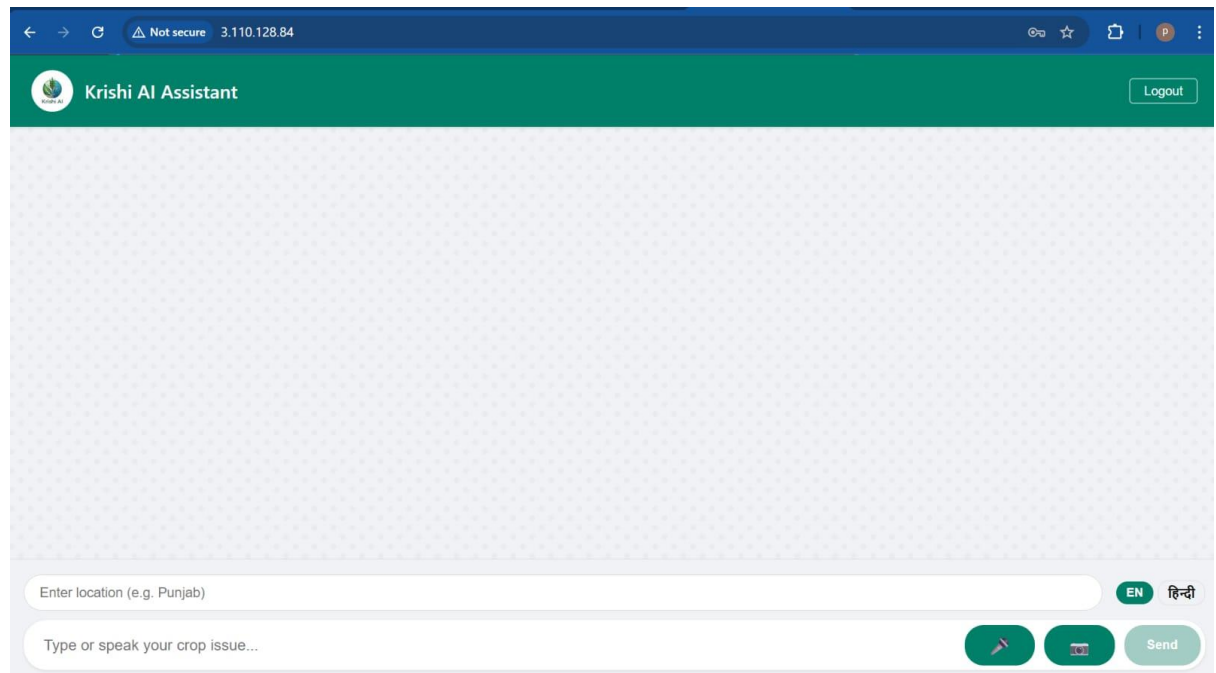
Login

Pods-

```
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$ microk8s kubectl get pods -n agriculture-app
```

NAME	READY	STATUS	RESTARTS	AGE
backend-6767efd77b-81c4g	1/1	Running	0	4d23h
backend-676c99c97d-p5hvd	0/1	Pending	0	44m
frontend-5869598f7f-tmb7m	1/1	Running	0	4d23h
frontend-5b47cbdf8-trffn	0/1	Pending	0	44m
portainer-6f8b6cfd69-kpnwf	1/1	Running	0	4d23h

Krishi interface-



2. Reports / dashboards / models

The IaC scripts successfully automated the full environment setup on target instances, including:

- Automated installation of Docker and Docker Compose on a fresh Ubuntu instance with a single script execution — no manual steps required.
- Docker Compose file serving as the definitive infrastructure definition — the entire multi-container environment is brought up with one command: `docker compose up`.
- Environment variables and API keys managed securely through `.env` files — no hardcoded secrets in any configuration.
- Repeatable deployment verified: the same scripts produce identical environments whether run locally or on AWS EC2.

Deployed docker images-

Image Name	Size	Status / Notes
agriculture-support-triage-agent-frontend	~63 MB	Running — Port 3000 → 80/tcp
agriculture-support-triage-agent-backend	~1.03 GB	Running — Port 8000 → 8000/tcp
portainer/portainer-ce	183 MB	Running — Container monitoring UI

Live endpoints-

- Frontend UI: <http://3.110.128.84> — accessible via browser
- Backend API Health Check: <http://3.110.128.84:8000/health>
- Swagger UI (API Testing): <http://3.110.128.84:8000/docs>
- Portainer(Dashboard): <http://3.110.128.84:9000>
 {user id-admin, Password-123456789012}

Api catalog

Category	Endpoint	Description
Authentication	POST /signup	Register a new farmer/user account
Authentication	POST /login	Login and receive JWT token
Core	POST /analyze	Upload crop image and receive AI diagnosis
Core	GET /market-price	Fetch current crop market price by location/type
Core	GET /insurance	Retrieve relevant insurance scheme information
System	GET /docs	Swagger UI for API testing and exploration

3. Key outcomes

- Infrastructure as Code achieved — the complete multi-service environment can be provisioned from scratch using automated scripts, eliminating manual configuration errors entirely.
- Repeatable deployments confirmed — identical environments reproduced on both local Docker and AWS EC2 using the same IaC scripts and Docker Compose definitions.
- CI/CD pipeline fully operational — Git-action automates every step from Git commit to running containers, reducing deployment time significantly.
- Groq Vision API integration enables real-time crop disease identification from uploaded images with treatment recommendations and urgency assessment.
- Stateless JWT-based authentication ensures the system can scale horizontally without session-state dependencies.
- Portainer successfully provides a web-based container management UI for monitoring service health without command-line access.

Conclusion

The IaC Provisioning for Agriculture System project successfully demonstrates how Infrastructure as Code principles can be applied to automate and simplify the deployment of a real-world, AI-powered agricultural support system. By replacing manual server configuration with script-based provisioning, the project eliminates environment inconsistencies and makes the entire system reproducible on any target machine — whether a local Docker environment or a cloud instance on AWS EC2.

The Krishi AI Assistant was built as a fully containerized application using Docker and Docker Compose, with each service (React frontend, FastAPI backend, Portainer monitoring) packaged and deployed as an independent container. Git-action CI/CD automation ensures that every code change flows seamlessly from Git commit to deployed container without human intervention.

Key learnings from this project:

- Understanding the value of IaC — writing infrastructure as scripts and configuration files ensures consistency, repeatability, and version-controlled deployments across all environments.
- Practical Docker skills — building Dockerfiles, managing image sizes, configuring port mappings, and using Docker Compose for multi-service orchestration.
- CI/CD pipeline design — setting up Git-action to automate build, test, and deploy cycles triggered by Git events.

- API integration with external AI services — securely integrating Groq Vision and LLM APIs with proper environment variable management.

- Stateless backend design — building scalable FastAPI services with JWT authentication that require no server-side session storage.
- Container security practices — isolating services, limiting port exposure, and storing secrets in .env files rather than code.

Overall, this project provided comprehensive, hands-on experience across the full DevOps lifecycle — from IaC-driven provisioning to containerized cloud deployment — while delivering a socially impactful tool for the Indian agricultural community.

Future Scope & Enhancements

Infrastructure & IaC

Introduce Terraform for declarative cloud infrastructure provisioning — automating VPC creation, EC2 instance setup, security groups, and networking on AWS.

- Add Ansible playbooks for configuration management — automating software installation and server configuration at a higher level than shell scripts.
- Migrate from Docker Compose to Kubernetes (K8s) for production-grade container orchestration, auto-scaling, and self-healing deployments.
- Add Redis as a separate container in the Docker Compose stack to cache AI responses, market data, and frequently accessed API results.

CI/CD & DevOps

- Implement automated testing stages (unit tests, integration tests) within the Git-action pipeline before deployment.
- Add Docker image scanning for vulnerabilities as part of the CI/CD pipeline.
- Set up multi-environment pipelines (dev → staging → production) with automated promotion gates.
- Use CDN (e.g., AWS CloudFront) for frontend static asset delivery to reduce latency.

Database & Data

- Migrate from SQLite to PostgreSQL for production-grade concurrent data access and reliability.
- Use Alembic for versioned database schema migrations during upgrades.
- Mount the database as a named Docker volume for persistent data across container restarts.
- Add automated database backup jobs as a lightweight scheduled container.

AI & Application

- Train a custom crop disease detection model and host it as a dedicated microservice to reduce dependency on external APIs.
- Add multilingual support (Hindi, Marathi, Telugu) to serve a wider farmer population.
- Develop a mobile application (React Native) for offline-capable field usage.
- Integrate real-time weather and soil health data APIs for more comprehensive crop advisory.

Security & Monitoring

- Deploy Prometheus and Grafana as monitoring containers for real-time metrics, dashboards, and automated alerting.
- Implement centralized logging using the ELK Stack (Elasticsearch, Logstash, Kibana).
- Add HTTPS/TLS via Let's Encrypt for secure communication across all endpoints. Implement role-based access control (RBAC) for farmer vs. admin differentiation